Підготовка до заняття 3

Інтеграція LLM: API та self-hosted

AI Engineering

На занятті 2 ми розібрали, як модель працює всередині. На занятті 3 вбудовуємо її в продукт: як підключати модель через API так, щоб код не падав, скільки коштує кожен запит, звідки береться затримка і коли варто запускати модель на своєму сервері. Цей документ допоможе прийти на заняття з розумінням ключових понять і з робочим середовищем, щоб на практиці писати код, а не налаштовувати ключі.

Після заняття ви зможете

- підключати LLM через API зі стійкими патернами: timeout, retry, structured output;
- рахувати вартість і latency запиту до того, як побачите рахунок;
- запускати модель локально через Ollama і чесно порівнювати її з API;
- аргументовано обирати модель під сценарій: якість, вартість, приватність, обмеження.

Як готуватися

1

~30 хв

Прочитайте глосарій на сторінках 2–4. Головне — зрозуміти механіку і приклади з цифрами.

2

~30 хв

Налаштуйте середовище за сторінками 5–6 і запустіть скрипт перевірки.

3

~15 хв

Дайте собі відповіді на питання для самоперевірки на сторінці 7.

4

~5 хв

Те, що лишилось незрозумілим, напишіть у тред Slack «Питання по теорії до заняття 3».

API і вартість запиту

Провайдерське API

OpenAI, Anthropic, Google, OpenRouter

Модель працює на серверах провайдера, а ваш код надсилає їй HTTP-запит зі списком повідомлень і отримує відповідь. Ви не керуєте залізом, платите за кожен токен і залежите від доступності сервісу та його лімітів.

→ Плюс: найсильніші моделі без власних серверів. Мінус: дані йдуть до третьої сторони, а ціна росте разом із трафіком.

OpenAI-сумісний API і base_url

Формат запиту OpenAI (`/v1/chat/completions`) став фактичним стандартом. Його підтримують OpenRouter, Ollama, vLLM, LM Studio та інші. Тому той самий код працює з різними моделями: змінюються лише адреса сервера (`base_url`), ключ і назва моделі.

→ Саме на цьому побудована практика заняття: одна задача — три реалізації через заміну двох рядків.

Вхідні та вихідні токени

Провайдер рахує окремо токени, які ви надіслали (input: system prompt, історія, документи), і токени, які згенерувала модель (output). Вихідні зазвичай у 3–5 разів дорожчі, бо генеруються по одному, а вхідні обробляються паралельно за один прохід (prefill, заняття 2).

→ gpt-4o-mini: $0.15 за 1 млн вхідних і $0.60 за 1 млн вихідних токенів. Актуальні ціни — на openrouter.ai/models.

Tokenomics: чому діалог дорожчає з кожним повідомленням

Модель нічого не пам'ятає між запитами, тому кожен новий запит несе всю історію чату. Приклад: system prompt 500 токенів, повідомлення користувача 100, відповідь моделі 200.

Хід діалогу	1	2	3	…	10	Разом за 10 ходів
Вхідні токени запиту	600	900	1 200	…	3 300	19 500

Нові тексти за 10 ходів — лише 1 500 токенів (system prompt і 10 повідомлень), а оплачуємо 19 500 вхідних, тобто в 13 разів більше. Вартість діалогу росте квадратично від його довжини.

→ 10 000 таких діалогів на день на gpt-4o-mini ≈ $41 на день, або ~$1 240 на місяць. Звідси прийоми: обрізати або стискати історію, скорочувати system prompt, кешувати.

Prompt caching

Якщо початок запиту (system prompt, інструкції, довгий документ) повторюється, провайдер може не обробляти його заново і бере за ці токени знижку. Кеш працює лише для однакового префікса, тому незмінне ставимо на початок, а змінне — в кінець.

Latency і надійність

Latency, TTFT і швидкість генерації

Час відповіді складається з двох частин. TTFT (time to first token) — мережа, черга на сервері провайдера і prefill вашого промпту. Далі модель генерує вихідні токени з певною швидкістю (tokens per second).

```
час відповіді ≈ TTFT + вихідні токени ÷ швидкість генерації
```

→ TTFT 0.4 с, відповідь 300 токенів при 60 токенах/с: 0.4 + 5 = 5.4 с. Скоротити відповідь удвічі дасть −2.5 с, скоротити промпт удвічі — частки секунди. Тому довжина виходу майже завжди важливіша за довжину входу.

Streaming

Відповідь приходить частинами по мірі генерації (`stream=True`). Загальний час не змінюється, але користувач бачить перші слова через TTFT, а не через 5 секунд.

→ Потрібен для чатів. Не потрібен для фонової обробки, де результат читає код.

Timeout

Максимальний час очікування відповіді. Без нього запит до перевантаженого провайдера може «висіти» хвилинами і блокувати ваш сервіс.

→ У SDK OpenAI: `OpenAI(timeout=30)` . Значення підбирають за реальною latency моделі з запасом.

Rate limit і retry з backoff

Провайдер обмежує кількість запитів і токенів за хвилину. Коли ліміт перевищено, він повертає помилку `429` . Тимчасові збої — це також `500` , `502` , `503` і timeout. На них правильна реакція — повторити запит, щоразу чекаючи довше: 1 с, 2 с, 4 с… плюс випадкова добавка (jitter), щоб тисячі клієнтів не повторили одночасно.

Помилка	Що означає	Retry?
429	перевищено ліміт	так, з backoff
500 / 502 / 503, timeout	збій на стороні провайдера	так, 2–3 спроби
400	помилка у вашому запиті	ні — виправляти код
401 / 403	ключ невірний або без доступу	ні

→ SDK OpenAI вже робить 2 повтори з backoff за замовчуванням (`max_retries`). На занятті подивимось, де цього недостатньо.

Structured output у продакшені

Нагадування з заняття 2: JSON Schema змушує модель повертати потрібну структуру. Але правильна структура ще не означає правильні дані. Тому відповідь завжди проходить валідацію в коді (Pydantic), а на помилку валідації є план: повторити запит, перейти на іншу модель або позначити запис для перевірки людиною.

→ Дешеві й локальні моделі частіше ламають формат — на практиці порахуємо, наскільки.

Self-hosted і вибір моделі

Self-hosted модель і Ollama

Модель з відкритими вагами (Llama, Qwen, Gemma, Mistral) працює на вашому залізі: ноутбуці, сервері компанії чи орендованому GPU. Ollama — найпростіший спосіб це зробити: одна команда завантажує модель, і вона одразу доступна через OpenAI-сумісний API на `localhost:11434`.

→ Перший запит після запуску повільний: модель вантажиться в пам'ять. У нас llama3.2:3b на MacBook відповіла за 5.7 с вперше і за 0.1 с наступного разу. Заміряючи latency, відкидайте перший запит.

Коли self-hosted виправданий

Дані не можна передавати третій стороні (медицина, банки, договори клієнтів).

Великий стабільний потік однотипних запитів, з яким справляється мала модель.

Потрібна робота без інтернету або повний контроль над версією моделі.

Коли це ілюзія економії

Сервер з GPU коштує грошей щогодини, навіть коли запитів немає.

До ціни заліза додаються налаштування, моніторинг, оновлення і час інженера.

Мала модель на складній задачі дає більше помилок, і їх виправлення теж коштує.

Порахуйте на прикладі

Орендований GPU рівня L4 / A10 — умовно $0.7–1 за годину, тобто $500–720 на місяць цілодобової роботи. За $720 через API gpt-4o-mini можна обробити приблизно 4,8 млрд вхідних токенів. Якщо ваш сервіс споживає 50 млн токенів на місяць, API обійдеться в ~$10, а сервер — у ті самі $720.

→ Self-hosted стає дешевшим лише при великому і рівномірному навантаженні. Для більшості MVP API вигідніший, а локальна модель потрібна з міркувань приватності, а не ціни.

Вибір моделі під сценарій: чотири питання

Критерій	Що питаємо
Якість	Чи справляється модель на ваших 20–50 реальних прикладах? Не на бенчмарках.
Вартість	Скільки коштує один запит × очікувана кількість запитів на місяць?
Приватність	Які дані йдуть у запит і чи можна їх відправляти цьому провайдеру?
Обмеження	Latency, rate limits, розмір контексту, підтримка structured output.

→ Відповідь рідко буде «одна модель на все». Частіше: дешева модель для простих запитів і сильна для складних.

НАЛАШТУВАННЯ СЕРЕДОВИЩА

Зробіть це до заняття

На практиці ви запускатимете один скрипт з API провайдера і з локальною моделлю. Налаштування займає ~30 хв, і на занятті на нього часу не буде.

1. Python і бібліотеки

Потрібен Python 3.10 або новіший (`python3 --version`). Встановіть бібліотеки:

```
pip install openai python-dotenv pydantic
```

2. Ключ OpenRouter у файлі .env

Ключ з заняття 2 підійде. На балансі достатньо кількох доларів, практика коштуватиме центи. У папці проєкту створіть два файли:

```
# .env — ключ тут, а не в коді
OPENROUTER_API_KEY=sk-or-...

# .gitignore — щоб ключ не потрапив у репозиторій
.env
```

3. Ollama і модель

Встановіть Ollama з **ollama.com/download** (macOS, Windows, Linux) і завантажте модель заздалегідь: це 2 ГБ, на занятті мережа буде перевантажена. Потрібно від 8 ГБ оперативної пам'яті.

```
ollama pull llama3.2:3b
ollama run llama3.2:3b "Привіт"
```

Немає можливості встановити локально?

Пройдіть практику в Google Colab, як на занятті 2: ключ у Secrets, Ollama запускається прямо в ноутбуці (код — у конспекті заняття 2, розділ «Мала чи велика, локальна чи провайдерська»). Але локальне середовище краще: далі в курсі ми працюватимемо зі скриптами і репозиторієм, а не лише з клітинками Colab.

НАЛАШТУВАННЯ СЕРЕДОВИЩА

Перевірте, що все працює

Скрипт надсилає однакове питання в OpenRouter і в локальну Ollama тим самим кодом. Різниця лише в `base_url`, ключі і назві моделі — так буде влаштована і практика на занятті.

4. Скрипт перевірки check.py

```
import os, time
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()  # бере OPENROUTER_API_KEY з файлу .env
QUESTION = "Назви столицю України одним словом."

TARGETS = [
    # назва, base_url, ключ, модель
    ("OpenRouter", "https://openrouter.ai/api/v1",
     os.getenv("OPENROUTER_API_KEY"), "openai/gpt-4o-mini"),
    ("Ollama", "http://localhost:11434/v1",
     "ollama", "llama3.2:3b"),
]

for name, base_url, key, model in TARGETS:
    if not key:
        print(f"{name}: немає OPENROUTER_API_KEY у .env")
        continue
    client = OpenAI(base_url=base_url, api_key=key,
                    timeout=60, max_retries=2)
    start = time.perf_counter()
    try:
        r = client.chat.completions.create(
            model=model,
            messages=[{"role": "user", "content": QUESTION}],
            temperature=0,
        )
    except Exception as e:
        print(f"{name}: помилка {type(e).__name__}: {e}")
        continue
    sec = time.perf_counter() - start
    u = r.usage
    answer = r.choices[0].message.content.strip()
    print(f"{name} | {model} | {sec:.1f} с | "
          f"токени: вхід {u.prompt_tokens}, вихід {u.completion_tokens} | {answer}")
```

Що побачите

рядок Ollama — реальний запуск на MacBook

```
OpenRouter | openai/gpt-4o-mini | 0.? с | токени: вхід ?, вихід ? | Київ
Ollama | llama3.2:3b | 5.7 с | токени: вхід 36, вихід 5 | Київ.
```

Цифри у вас будуть інші. Перший запит до Ollama повільний, бо модель вантажиться в пам'ять: запустіть скрипт удруге і порівняйте.

ЗАНЯТТЯ 3

Як пройде заняття

Роадмап	Де ми в курсі і що забираємо з заняття 2.
Теорія	Патерни інтеграції, tokenomics, latency, self-hosted. Акцент на питаннях, які ви надішлете в тред заздалегідь.
Практика	Скрипт класифікує 5 звернень клієнтів і повертає JSON. Ви міняєте `base_url` і модель, запускаєте і пишете в чат: модель, час, вартість, скільки з 5 відповідей — валідний JSON. Разом зведемо таблицю трейд-офів.
Зв'язок з кейсом 1	Крок 2 кейсу: друга модель і заміри якості, вартості й latency на задачі вашого проєкту.
Q&A	Питання з чату заняття і з треду.

Як працюємо з питаннями

- Питання по теорії пишіть до заняття в тред Slack «Питання по теорії до заняття 3». Якщо питання вже є, поставте йому реакцію: так видно, що цікаво більшості.
- На занятті питання пишемо в чат. Відповіді — на короткій паузі після теорії і в Q&A наприкінці.
- Технічні проблеми з середовищем, питання про ваш робочий кейс і організаційні питання розбираємо в Slack.

Питання для самоперевірки

Спробуйте відповісти до заняття. Розберемо їх у теорії.

01	Чому 10-те повідомлення в чаті коштує дорожче за перше, хоча воно такої ж довжини?
02	Що сильніше зменшить час відповіді: скоротити промпт удвічі чи відповідь удвічі? Чому?
03	На які помилки API варто робити retry, а на які — ні?
04	Локальна модель відповідає за 6 с, а API — за 1.5 с. Назвіть дві можливі причини.
05	Коли self-hosted справді дешевший за API, а коли лише здається дешевшим?
06	Які дані з вашого курсового проєкту не можна відправляти сторонньому провайдеру?

ДЛЯ ПОГЛИБЛЕННЯ

Три джерела до теми

Усі англійською. Не обов'язково до заняття: читайте, коли дійдете до відповідного кроку кейсу.

01 Latency optimization

документація · OpenAI

Сім принципів зменшення latency: швидша модель, менше вихідних і вхідних токенів, менше запитів, паралельність, streaming і не використовувати LLM там, де вистачить коду. Короткий текст, який варто знати кожному, хто вбудовує LLM у продукт.

platform.openai.com/docs/guides/latency-optimization

02 How to handle rate limits

приклади коду · OpenAI Cookbook

Що таке rate limits, як виглядає помилка 429 і як писати retry з експоненційним backoff. Приклади на Python, які можна взяти у свій проєкт.

cookbook.openai.com/examples/how_to_handle_rate_limits

03 Ollama: OpenAI compatibility

документація · Ollama

Як підключити локальну модель тим самим кодом, що й OpenAI: які ендпоінти і параметри підтримуються, а які ні. Знадобиться на кроці 2 кейсу 1.

docs.ollama.com/api/openai-compatibility

Ціни і моделі

Актуальні ціни за вхідні й вихідні токени, розмір контексту і підтримку structured output для сотень моделей зручно порівнювати на **openrouter.ai/models**. Ціни змінюються, тому перед розрахунками перевіряйте їх там.